JZ-HDL TESTBENCH SPECIFICATION

State: Beta — Version: 0.1.8

Contents

1. OVERVIEW

This specification defines the testbench language and simulation semantics for JZ-HDL. Testbenches are verification constructs that drive and observe synthesizable modules in a cycle-accurate simulation environment. They are strictly non-synthesizable and exist in a separate namespace from RTL modules.

JZ-HDL simulation operates at the **cycle level**. There are no absolute time units, no propagation delays, and no inertial/transport delay models. All timing is expressed in terms of clock cycles. This is a deliberate constraint: if a design compiles, its structural correctness is guaranteed by the compiler; simulation verifies **functional behavior** within that structure.

Design Principles: - **Cycle-relative timing:** All time advancement is expressed in clock cycles, never in absolute time units. - **Explicit synchronization:** Stimulus changes occur only at designated `@update` points between clock phases. There is no ambiguity about when an input changed relative to the clock. - **Deterministic ordering:** The sequence of operations within a test is strictly linear. There is no concurrent stimulus application, no fork/join, and no event-driven scheduling. - **RTL isolation:** Testbench constructs cannot appear inside `@module` or `@project`. RTL constructs cannot appear inside `@testbench`. The compiler enforces this boundary at parse time. - **No delays:** There are no `#5`, `after`, `wait`, or inertial/transport delay semantics. If users need analog timing analysis, they are in the wrong language.

Relationship to `@simulation`:

`@testbench` and `@simulation` are two **independent execution models**. They share the same RTL evaluation semantics (combinational settling, NBA for synchronous updates, reset behavior) but differ fundamentally in how time advances:

- `@testbench` is **cycle-stepped**: clocks advance only when explicitly commanded by `@clock` directives, one clock at a time. There is no notion of absolute time, clock periods, or simultaneous multi-clock advancement.
- `@simulation` is **time-based**: clocks run continuously with defined periods, multiple clocks toggle independently via GCD-based tick scheduling, and time advances via `@run` directives in absolute units.

Neither model is a superset of the other. A testbench implementation is not required to use the simulation engine internally, and vice versa. The testbench model is intentionally simpler — it trades multi-clock fidelity for deterministic, easy-to-reason-about test sequences.

2. SIMULATION EXECUTION MODEL

This section defines the normative simulation semantics that underpin testbench execution. These rules are not visible to the designer but are mandatory for any conforming JZ-HDL simulator.

2.1 Simulation Phases

Each simulation step proceeds through the following phases in strict order:

```
Simulation Step:
  1. STIMULUS PHASE      – Apply external stimulus (@setup or @update values)
  2. COMBINATIONAL PHASE – Evaluate all ASYNCHRONOUS logic to a fixed point
  3. SAMPLE PHASE        – Sample clock edges and reset signals
  4. SEQUENTIAL PHASE    – Apply all SYNCHRONOUS updates (NBA semantics)
  5. SETTLE PHASE        – Re-evaluate ASYNCHRONOUS logic to a fixed point
                           (sequential outputs may feed combinational inputs)
  6. OBSERVE PHASE     – All signals are stable; assertions and monitors execute
```

Phases 2-5 repeat for each clock edge within a `@clock` directive. For a `@clock(clk, cycle=N)` directive, the simulator executes 2N edge transitions (N rising edges + N falling edges), running phases 2-5 at each edge where synchronous logic is sensitive.

2.2 Combinational Settling

During the COMBINATIONAL PHASE and SETTLE PHASE, the simulator evaluates all ASYNCHRONOUS logic to a **fixed point** (quiescence). The settling algorithm operates as follows:

1. Evaluate all ASYNCHRONOUS blocks across the entire elaborated hierarchy (parent module, then recursively all child instances). This includes:
   - All ASYNCHRONOUS assignments (combinational wire drivers).
   - Latch transparent-state updates — latches whose enable is active evaluate their data input and update their output as part of the combinational evaluation.
   - Asynchronous memory reads — when a memory port's address changes, the read data output updates combinationally (see JZ-HDL specification Section 7.3).
2. After one complete evaluation pass, check whether any signal value changed compared to its value before the pass.
3. If any value changed, repeat from step 1 (a new **delta cycle**).
4. If no values changed, the logic has reached quiescence. Settling is complete.

Iteration limit. If quiescence is not reached within **100 delta cycles**, the simulator reports a **combinational loop runtime error** (SE-001) and aborts the current test case. The compiler's flow-sensitive static analysis (JZ-HDL specification Section 12.2) catches most combinational loops at compile time. The simulation limit is a dynamic safety net for cases the static analysis cannot prove, such as loops that depend on runtime-dependent control flow.

Convergence scope. The convergence check compares all signal values (wires, ports, latches) in the top-level DUT context. Child instance signals are settled recursively but convergence is determined by whether the parent-visible signals have stabilized. Since child outputs propagate to parent signals during each pass, a child-only oscillation will manifest as a parent signal change.

2.3 Sequential Update Semantics

SYNCHRONOUS updates use **non-blocking assignment** (NBA) semantics:

1. All RHS expressions in SYNCHRONOUS blocks are evaluated using current (pre-update) register values.
2. All register updates are applied simultaneously after all RHS evaluations complete.
3. After updates, ASYNCHRONOUS logic is re-evaluated to a fixed point (SETTLE PHASE).

This matches the hardware behavior of edge-triggered flip-flops: all registers sample their inputs at the clock edge and update simultaneously.

2.4 Reset Semantics

Reset behavior in simulation follows the module's declared reset configuration:

Immediate Reset (`RESET_TYPE=Immediate`): - When the reset signal meets the `RESET_ACTIVE` condition, all registers in the SYNCHRONOUS block are forced to their declared reset values **immediately**, bypassing the clock. - This takes effect during the SAMPLE PHASE. After reset values are applied, ASYNCHRONOUS logic re-evaluates (SETTLE PHASE). - Immediate reset overrides any pending SYNCHRONOUS updates.

Clocked Reset (`RESET_TYPE=Clocked`): - When the reset signal meets the `RESET_ACTIVE` condition at a clock edge, all registers are loaded with their declared reset values at that edge. - This is evaluated as part of the normal SEQUENTIAL PHASE, with reset taking priority over all SYNCHRONOUS assignments.

2.5 X and Z Semantics in Simulation

Simulation follows the same x and z semantics as the JZ-HDL specification (Sections 1.6 and 2.1). The rules below are not independent — they restate the compile-time invariants and define the simulator's dynamic enforcement of those invariants.

2.5.1 X Is Not a Runtime Value The value x is a **compile-time concept only**. It represents an intentional don't-care bit in CASE/SELECT pattern matching (see JZ-HDL specification Section 2.1). The compiler's observability rule guarantees that x bits are structurally masked before reaching any observable sink.

x never appears during simulation. All storage (registers, latches, memory) initializes with random `0`/1 bits (Section 2.6). All wires and ports carry only `0`, 1, or `z`. If the simulator's internal computations ever produce an x state, this indicates a z value reached a non-tristate expression — which is a runtime error (Section 2.5.3).

2.5.2 Z in Simulation High-impedance (z) represents the absence of a driven value. In simulation, z appears only on nets and ports that participate in tri-state logic. The simulator applies the same resolution rules defined in JZ-HDL specification Section 1.6.3:

- A driver assigning z is electrically disconnected and does not contribute a logic value.
- When multiple drivers exist on a net, tri-state resolution applies per bit using the resolution table from Section 1.6.3 of the JZ-HDL specification.

- Registers and latches cannot store or produce z (JZ-HDL specification Section 1.6.6).
- The compiler's multi-driver validity rules (JZ-HDL specification Section 1.6.4) structurally prove that at most one driver is active per execution path. The `0`/1 conflict case in the resolution table is prohibited at compile time and cannot occur in simulation.
- If a net resolves to all-z and is read in a non-tristate context, the simulator reports a **runtime error** (floating net read). The compiler's static analysis catches most such cases at compile time; simulation provides a dynamic safety net for cases involving runtime-dependent control flow.

2.5.3 Runtime Error: Z in Non-Tristate Expressions The compiler's observability rule (JZ-HDL specification Section 1.6.7) structurally prevents z from reaching registers, output ports, and other observable sinks. However, the compile-time check operates at the expression level and does not perform deep dataflow tracking across intermediate wires. In rare cases involving runtime-dependent control flow, z may reach an expression at simulation time despite passing compile-time checks.

When z appears as an operand in a non-tristate expression (arithmetic, comparison, conditional, bitwise logic, or reduction), the simulator reports a **runtime error** and aborts the current test case. The error report includes the signal name, the z bit positions, and the source location.

2.5.4 Observability at Assertions If an `@expect_equal` or `@expect_not_equal` assertion observes a signal containing z bits, it is a **runtime error**. The simulator aborts the current test case and reports the z bit positions. Assertions demand fully determined values (`0` or 1 only).

Exception: `@expect_tristate` (Section 6.8) specifically tests for the z state rather than treating z as an error. It asserts that all bits of a signal are in the high-impedance state.

2.6 Storage Initialization

All REGISTER, LATCH, and MEM storage elements initialize with **random but determinate** bits (each bit is either `0` or 1, chosen randomly) at the start of each test case or simulation run. This is independent of their declared reset values.

The declared reset value is applied **only** when: - The module's RESET signal meets the RESET_ACTIVE condition. - At the appropriate time per RESET_TYPE (Immediate or Clocked).

Important: A register's declared reset value is never applied automatically at instantiation or power-on. Reset values take effect only when the module's reset signal is explicitly asserted during simulation. If a module has no reset port, or if the testbench never asserts reset, all registers retain their random power-on values for the entire simulation — exactly as in real hardware. The test must explicitly assert and release reset to bring registers to known values.

Before RESET is asserted, reading from uninitialized storage yields random bits — exactly as in real hardware. The simulator must use a different random seed for each test run by default, with an option to specify a fixed seed for reproducibility.

2.7 Stable Hierarchical Names

Every signal in the simulation hierarchy has a **stable, deterministic hierarchical name** of the form:

```
<instance_name>.<signal_name>
<instance_name>.<sub_instance>.<signal_name>
<instance_name>.<mem_name>.<port_name>
```

The elaboration process must produce identical hierarchical paths across compilations given the same source. This is critical for: - Waveform dumps (VCD/FST) - Signal monitoring expressions - Assertion references - Debugging reproducibility

3. TESTBENCH STRUCTURE

3.1 Testbench Declaration

Syntax:

```
@testbench <module_name>
    @import "<path>";      // optional: import modules
    BUS <name> { ... }     // optional: BUS definitions

    CLOCK { ... }
    WIRE { ... }

    TEST "<description>" { ... }
    ...
@endtb
```

- <module_name> must refer to a previously defined @module that is in scope (via @import or within the same compilation unit).
- A testbench may contain zero or more @import directives, zero or more BUS definitions, zero or more CLOCK declarations, zero or more WIRE declarations, and one or more TEST blocks.
- @import directives and BUS definitions must appear before CLOCK, WIRE, and TEST blocks.
- The CLOCK and WIRE blocks are testbench-level declarations shared across all TEST blocks.
- Multiple @testbench blocks may target the same module. Each is independent.

3.2 File Organization

Testbench files use the same .jz extension as RTL files. A single file may contain: - @import directives (to bring modules into scope) - @global blocks (shared constants) - BUS definitions (shared bus type definitions) - One or more @testbench blocks

@import directives, @global blocks, and BUS definitions may appear either at file level (before @testbench) or inside a @testbench block. When placed inside @testbench, imported modules and BUS definitions are lifted to file scope and are visible to all testbenches in the file.

A file may **not** contain both @module/@project definitions and @testbench blocks. The compiler rejects such files with a parse error. This enforces the RTL/verification boundary.

4. CLOCK BLOCK

4.1 Declaration

Syntax:

```
CLOCK {
    <clock_id>;
    ...
}
```

Each clock declaration creates a named clock signal that the testbench controls. Clocks are not free-running — they advance only when explicitly commanded by `@clock` directives within a test case.

4.2 Clock Behavior

- A clock is a width-1 signal initialized to `1'b0`.
- Clocks are not assignable in `@setup` or `@update` — they are driven exclusively by `@clock` directives.
- The `@clock` directive toggles the clock for the specified number of complete cycles, where each cycle consists of one rising edge followed by one falling edge.
- Between `@clock` directives, the clock holds its last value and does not toggle.
- If `@clock` is never called for a declared clock, that clock remains at `1'b0` for the entire test.

5. WIRE BLOCK

5.1 Declaration

Syntax:

```
WIRE {
    <wire_id> [<width>];
    BUS <bus_id> [<count>] <group_name>;
    ...
}
```

Testbench wires are the signals used to drive module inputs and observe module outputs. They follow the same width declaration syntax as module-level `WIRE` blocks.

5.2 BUS Wire Declarations

A `BUS` wire declaration expands a named BUS definition into individual testbench wires. The syntax is:

```
BUS <bus_id> [<count>] <group_name>;
```

- `<bus_id>` must refer to a `BUS` definition in scope (at file level or inside the `@testbench` block).
- `[<count>]` is an optional array count. When present, wires are named `<group_name><N>_<signal>` for each element `N` from `0` to `count−1`. When omitted, wires are named `<group_name>_<signal>`.
- Each BUS signal becomes an individual testbench wire with the width declared in the BUS definition.

Example:

Given a BUS definition:

```
BUS PARALLEL_BUS {
    OUT   [16]    ADDR;
    OUT   [1]     CMD;
    OUT   [1]     VALID;
    INOUT [16]    DATA;
    IN    [1]     DONE;
}
```

The declaration `BUS PARALLEL_BUS [2] src;` expands to:

```
src0_ADDR [16];  src0_CMD [1];  src0_VALID [1];  src0_DATA [16];  src0_DONE [1];
src1_ADDR [16];  src1_CMD [1];  src1_VALID [1];  src1_DATA [16];  src1_DONE [1];
```

5.3 Semantics

- Testbench wires are **procedural drivers**, not combinational nets. They hold their assigned value until explicitly changed by an `@update` block.
- A testbench wire connected to a module `IN` port acts as a stimulus driver.

- A testbench wire connected to a module `OUT` port acts as an observation point. The wire reflects the module's output value after the most recent clock advancement or combinational settling.
- A testbench wire connected to a module `INOUT` port acts as both driver and observer. Its driven value participates in tri-state resolution with the module's internal driver.
- Testbench wires initialize to `0` (all bits zero) at the start of each test.

Example:

```
WIRE {
    rst_n [1];
    data_in [8];
    result [8];
    valid [1];
    BUS PARALLEL_BUS [2] src;
    BUS PARALLEL_BUS [6] tgt;
}
```

6. TEST BLOCK

6.1 Declaration

Syntax:

```
TEST "<description>" {
    @new <instance_name> <module_name> {
        <port_id> [<width>] = <wire_id | clock_id>;
        ...
    }

    @setup { ... }

    <sequence of @clock, @update, @expect_equal, @expect_not_equal, @expect_tristate di
}
```

Each TEST block is an independent test case. Test cases do not share state — each test begins with a fresh module instance and fresh wire values.

- `<description>` is a string literal used for identification in test output and failure reports.
- A test block must contain exactly one `@new` instantiation and exactly one `@setup` block.
- The `@setup` block must appear after `@new` and before any `@clock`, `@update`, or `@expect` directives.

6.2 Module Instantiation

Syntax:

```
@new <instance_name> <module_name> {
    <port_id> [<width>] = <wire_id | clock_id>;
    BUS <bus_id> <port_prefix> = <wire_prefix>;
    ...
}
```

The `@new` directive creates an instance of the module under test and connects its ports to testbench clocks and wires. It follows all the same connection rules as `@new` in the JZ-HDL specification (Section 4.9), including OVERRIDE.

- `<instance_name>` is the local name for the instance within this test.
- `<port_id>` must match a declared port on the target module.
- `[<width>]` must match the port's declared width.
- The right-hand side must be a testbench `CLOCK` or `WIRE` identifier.
- All module ports must be connected. Unconnected ports are a compile error.

6.2.1 BUS Port Bindings BUS ports may be connected using a shorthand that binds all expanded BUS signals at once:

```
BUS <bus_id> <port_prefix> = <wire_prefix>;
```

- `<bus_id>` must refer to a `BUS` definition in scope.

- <port_prefix> is the BUS port name as declared on the module (e.g., src). The binding matches all DUT port signals whose name starts with <port_prefix> followed by a digit or underscore (e.g., src0_ADDR, src0_CMD, src1_VALID).
- <wire_prefix> is the corresponding testbench wire group name. For each matched DUT port signal, the port prefix is replaced with the wire prefix to find the testbench wire (e.g., src0_ADDR maps to wire src0_ADDR when both prefixes are src).

BUS ports may also be connected by binding each BUS signal individually, following the same expansion rules as module-level BUS instantiation.

Examples:

```
// Individual bindings
@new dut counter {
    clk [1] = clk;
    rst_n [1] = rst_n;
    count [8] = count;
}

// BUS shorthand bindings
@new dut arbiter {
    clk [1] = clk;
    rst_n [1] = rst_n;
    map_config [48] = map_config;
    BUS PARALLEL_BUS src = src;
    BUS PARALLEL_BUS tgt = tgt;
}
```

6.3 @setup

Syntax:

```
@setup {
    <wire_id> <= <literal>;
    ...
}
```

The @setup block establishes the initial electrical state of all testbench wires before simulation begins. It executes once at the start of the test, at simulation time zero, before any clock edges.

- Every testbench wire that drives a module input should be assigned an initial value.
- The <= operator is used for all assignments within @setup.
- Assignments in @setup take effect simultaneously (they are not ordered).
- Any wire not assigned in @setup is implicitly initialized to 0.
- Clock signals may not be assigned in @setup; they initialize to 1'b0 automatically.

Example:

```
@setup {
    rst_n <= 1'b0;
    data_in <= 8'h00;
```

```
}
```

6.4 @clock

Syntax:

```
@clock(<clock_id>, cycle=<count>)
```

The `@clock` directive advances simulation time by toggling the named clock for `<count>` complete cycles.

- `<clock_id>` must refer to a clock declared in the testbench `CLOCK` block.
- `<count>` must be a positive integer or `CONST` expression.
- Each cycle consists of one full period of the clock waveform (rising edge, high phase, falling edge, low phase), starting from the clock's current state.
- After the directive completes, all synchronous logic in the module under test has been evaluated for each clock edge, and all combinational logic has settled.
- Multiple `@clock` directives may appear in sequence, potentially for different clocks.

Example:

```
@clock(clk, cycle=10)    // Advance 10 complete clock cycles
@clock(clk, cycle=1)     // Advance 1 cycle
```

6.5 @update

Syntax:

```
@update {
    <wire_id> <= <literal_or_expression>;
    ...
}
```

The `@update` block changes the values of testbench wires at a defined synchronization point between clock phases.

- The `@update` block pauses clock toggling, applies the new wire values, allows combinational logic to settle, and then yields control to the next directive.
- Only testbench `WIRE` identifiers may be assigned in `@update`. Clock signals may not be reassigned.
- Assignments within a single `@update` block take effect simultaneously.
- An `@update` block may appear anywhere after `@setup` in the test sequence.

Simultaneous Assignment Semantics:

All assignments within a single `@update` block execute with atomic, simultaneous semantics: 1. Evaluate all RHS expressions using the current state (before any assignments take effect). 2. Atomically update all LHS targets with their evaluated values. 3. After all assignments complete, allow combinational logic to settle before the next directive.

This means cyclic dependencies within `@update` are well-defined (e.g., swap):

```
@update {
    a <= b;
    b <= a;
}
// Result: a and b exchange values
```

All RHS expressions read the pre-update values of all wires — not the values being assigned in the same block. This applies even when the same wire appears on both LHS and RHS:

```
@update {
    a <= a + 8'h01;
    b <= a;
}
// b receives the OLD value of a (before the increment).
// a receives old_a + 1.
// This is NOT sequential: b does not see the incremented value.
```

This semantics differs from module combinational logic, where cyclic dependencies are forbidden. In testbenches, `@update` blocks are procedural drivers, not combinational nets, and simultaneous assignment is safe and deterministic.

Example:

```
@update {
    rst_n <= 1'b1;
    data_in <= 8'hAB;
}
```

6.6 @expect_equal

Syntax:

```
@expect_equal(<signal>, <expected_value>)
```

Asserts that a signal's current value matches the expected value at the current simulation point.

- `<signal>` may be any testbench wire (including those connected to module outputs) or a hierarchical reference to an internal signal of the module under test.
- `<expected_value>` must be a sized literal or `CONST` expression.
- The comparison is performed after all combinational logic has settled following the most recent `@clock` or `@update` directive.
- If the assertion fails, the test case is marked as failed and the simulator reports a diagnostic.

Width Rule: The width of `<expected_value>` must exactly match the width of `<signal>`. Width mismatches are a compile error.

Assertion Values: `<expected_value>` must be a fully-determined literal (no `z` bits; `x` does not exist at runtime).

Module Output Observation: If a module output contains `z` (e.g., from a floating net or unresolved tristate), observing it in an assertion is a runtime error. The simulator aborts the test case with a diagnostic showing the `z` bit positions.

Hierarchical References: Hierarchical references access internal signals: `<instance_name>.<signal_id>`. The signal must be a declared REGISTER, WIRE, or LATCH in the target module. Nested references (e.g., `dut.sub.signal`) are supported.

Example:

```
@expect_equal(result, 8'hFF)
@expect_equal(valid, 1'b1)
@expect_equal(dut.internal_reg, 16'h0042)
```

6.7 @expect_not_equal

Syntax:

```
@expect_not_equal(<signal>, <value>)
```

Asserts that the signal's current value does **not** match the given value.

- Same rules as `@expect_equal` regarding signal references, width matching, and evaluation timing.

Example:

```
@expect_not_equal(result, 8'h00)
```

6.8 @expect_tristate

Syntax:

```
@expect_tristate(<signal>)
```

Asserts that all bits of the signal are currently in the high-impedance (z) state.

- `<signal>` may be any testbench wire (including those connected to module outputs) or a hierarchical reference to an internal signal.
- Unlike `@expect_equal` and `@expect_not_equal`, this directive takes only one argument — no expected value is needed since the expected state is always all-z.
- The check is performed after all combinational logic has settled following the most recent `@clock` or `@update` directive (same timing rules as other assertions).
- If the assertion fails (any bit is not z), the test case is marked as failed and the simulator reports a diagnostic showing the actual value.

Use case: Verifying tristate output behavior — that a module correctly drives its output to high-impedance when disabled or when its bus grant is inactive.

Example:

```
@expect_tristate(data_out)
```

7. STIMULUS PATTERNS

This section describes common stimulus patterns available within `@update` blocks.

7.1 Literal Assignments

The simplest stimulus form assigns a sized literal to a testbench wire:

```
@update {
    data_in <= 8'hFF;
    enable <= 1'b1;
}
```

7.2 Expression Assignments

Testbench `@update` blocks support the same expression syntax as ASYNCHRONOUS blocks (arithmetic, bitwise, concatenation, ternary), operating on testbench wire values:

```
@update {
    addr <= addr + 8'h01;
    data <= {upper_nibble, lower_nibble};
}
```

Note: Expressions in `@update` reference the **pre-update** values of all wires (simultaneous assignment semantics).

7.3 Global Constants

`@global` constants are available in testbench contexts:

```
@global CMD
    READ  = 2'b01;
    WRITE = 2'b10;
    IDLE  = 2'b00;
@endglob

@testbench memory_controller
    WIRE {
        cmd [2];
    }

    TEST "Write then read" {
        // ...
        @update {
            cmd <= CMD.WRITE;
        }
        @clock(clk, cycle=1)
        @update {
            cmd <= CMD.READ;
        }
```

```
    }
@endtb
```

7.4 @repeat

Syntax:

```
@repeat <count>
<body>
@end
```

The `@repeat` directive is a **pre-parser text expansion**. Before lexing or parsing, the compiler scans the source text for `@repeat N ... @end` blocks, duplicates the body N times, and replaces each standalone occurrence of the identifier `IDX` with the iteration index (0 through N-1).

- `<count>` must be a positive integer literal.
- `<body>` may contain any valid testbench content: `@clock`, `@update`, `@expect_equal`, `@expect_not_equal`, `@expect_tristate`, comments, or any other text.
- `IDX` is replaced on word boundaries only. It will not match inside identifiers like `INDEX` or `MY_IDX_VAR`.
- Nesting is supported: an inner `@repeat` expands fully within each iteration of the outer `@repeat`.
- `@end` must not be confused with `@endmod`, `@endtb`, `@endsim`, or other compound closing directives. Only a standalone `@end` (not followed by alphabetic characters) closes a `@repeat` block.
- `@repeat` inside comments or string literals is ignored (not expanded).

Example — Multi-cycle clock advancement:

```
// Equivalent to writing @clock(clk, cycle=1) five times
@repeat 5
@clock(clk, cycle=1)
@end
```

Example — IDX substitution:

```
// Advance clock and check incrementing values
@repeat 4
@clock(clk, cycle=1)
@expect_equal(count, 8'hIDX)
@end
// Expands to:
// @clock(clk, cycle=1)
// @expect_equal(count, 8'h0)
// @clock(clk, cycle=1)
// @expect_equal(count, 8'h1)
// @clock(clk, cycle=1)
// @expect_equal(count, 8'h2)
// @clock(clk, cycle=1)
// @expect_equal(count, 8'h3)
```

Rules:

Rule	Description
RPT-001	`@repeat` requires a positive integer count
RPT-002	`@repeat` without matching `@end`

7.5 @print

Syntax:

```
@print("<format_string>", <arg1>, <arg2>, ...)
```

The `@print` directive outputs a formatted message to the simulator's standard output at the point in the test sequence where it appears.

- `<format_string>` is a string literal containing text and optional format specifiers.
- Arguments following the format string are testbench wire identifiers or hierarchical signal references, matched positionally to format specifiers.
- The message is printed after all combinational logic has settled following the most recent `@clock` or `@update` directive.
- `@print` may appear anywhere in the test sequence where `@expect_equal` is valid.

Format specifiers:

Specifier	Description
`%h`	Display the argument value in hexadecimal
`%d`	Display the argument value in decimal
`%b`	Display the argument value in binary
`%tick`	Display the current cycle count (no argument consumed)
`%ms`	Display the current simulation time in milliseconds (no argument consumed)

- `%tick` and `%ms` are **autonomous specifiers** — they do not consume an argument from the argument list.
- The number of non-autonomous format specifiers must match the number of arguments. A mismatch is a compile error.

Example:

```
@print("count = %h at cycle %tick", count)
@print("addr=%h data=%d", addr, data_out)
@print("cycle %tick: done")
```

7.6 @print_if

Syntax:

```
@print_if(<condition>, "<format_string>", <arg1>, <arg2>, ...)
```

The `@print_if` directive conditionally outputs a formatted message. The message is printed only if `<condition>` evaluates to a non-zero (truthy) value.

- `<condition>` is a testbench wire identifier or hierarchical signal reference. The condition is truthy if any bit is 1.
- The format string and arguments follow the same rules as `@print`.
- `@print_if` may appear anywhere `@print` is valid.

Example:

```
@print_if(valid, "data captured: %h at cycle %tick", data_out)
@print_if(error, "ERROR at cycle %tick: expected %h got %h", expected, actual)
```

Rules:

Rule	Description
PRT-001	Number of non-autonomous format specifiers must match the number of arguments
PRT-002	`@print` / `@print_if` may not appear inside `@setup` or `@update` blocks

8. EXECUTION MODEL

8.1 Testbench Execution Flow

A test case executes as a strictly ordered sequence of phases:

1. **Storage randomizes.** All REGISTER, LATCH, and MEM storage is filled with pseudo-random values derived from the simulation seed (Section 2.6). This models the indeterminate power-on state of real flip-flops and memory cells.
2. **Clocks hold at `1'b0`.** No clock has toggled yet. All declared clocks are at their initial low state.
3. **`@setup` executes.** Explicit initial wire values (e.g., `rst_n <= 1'b0`) are applied to the testbench wires. Any wire not assigned in `@setup` is implicitly initialized to `0`.
4. **Combinational logic settles.** Inputs propagate through the DUT, all ASYNCHRONOUS paths evaluate to a fixed point, and outputs propagate back to testbench wires.
5. **Test sequence begins.** Each subsequent directive executes in order:
   - `@clock` toggles the named clock for the specified cycles, evaluating synchronous and combinational logic at each edge.
   - `@update` applies new wire values and allows combinational logic to settle.
   - `@expect_equal` / `@expect_not_equal` / `@expect_tristate` samples the current signal value and checks the assertion.
6. **Completion.** After the last directive, the test result (pass or fail) is reported.

8.2 Timing Guarantees

- **Setup-hold compliance:** `@update` applies stimulus between clock edges, never coincident with an active edge. This guarantees that setup and hold timing requirements are met in simulation.
- **No race conditions:** Because stimulus changes (`@update`) and clock advancement (`@clock`) are never simultaneous, there is no ambiguity about signal ordering.
- **Combinational settling:** After every `@clock` and `@update`, all combinational paths are fully evaluated before the next directive executes. Assertions and monitors always observe a stable state.

8.2.1 Determinism Guarantee

Given identical input source files and seed, all simulation output — assertion results, pass/fail verdicts, and diagnostic messages — must be bit-identical across runs, across platforms, and across conforming implementations.

This is a normative requirement. There is no implementation-defined ordering, no thread-dependent scheduling, and no platform-dependent evaluation. Every aspect of simulation is fully determined by the source text and the seed value:

- Storage randomization is derived solely from the seed via a specified PRNG algorithm (Section 2.6).
- Statement evaluation order is fixed by the source-level ordering of directives, blocks, and assignments.
- Combinational settling follows a deterministic iteration order (Section 2.2).
- Assertion comparison uses exact bit-vector equality with no floating-point arithmetic.

If two conforming simulators produce different results for the same source and seed, at least one has a bug.

8.3 Multiple Clocks

When a testbench declares multiple clocks, `@clock` directives advance only the named clock. Other clocks remain held at their current level. To advance multiple clocks, issue separate `@clock` directives in the desired order.

The testbench does not support simultaneous advancement of multiple clocks in a single directive. Multi-clock designs must be tested by interleaving `@clock` calls for each domain.

Limitation: Because clocks advance sequentially rather than concurrently, the testbench model does not reproduce real-world phase relationships between clock domains. Two `@clock` directives issued back-to-back do not overlap in time — the second clock is frozen while the first advances, and vice versa. This model verifies **functional correctness** (data integrity across domains, handshake protocol compliance) but not cycle-accurate multi-clock timing. For timing-accurate multi-clock verification with realistic phase alignment, use `@simulation` with defined clock periods.

Example:

```
CLOCK {
    fast_clk;
    slow_clk;
}

TEST "Multi-clock domain" {
    // ...
    @clock(fast_clk, cycle=4)   // 4 fast cycles (slow_clk frozen)
    @clock(slow_clk, cycle=1)   // 1 slow cycle (fast_clk frozen)
    @clock(fast_clk, cycle=4)   // 4 more fast cycles
}
```

9. FAILURE REPORTING

9.1 Assertion Failure Format

When an `@expect_equal` or `@expect_not_equal` assertion fails, the simulator produces a diagnostic report containing:

- The test case description string.
- The directive that failed and its source location.
- The expected value and the actual value, displayed in the literal's declared base.
- The current simulation cycle count.
- Relevant internal state of the module under test at the point of failure.

Example Output:

```
FAIL: "Increment and Wrap"
  @expect_equal(result, 8'h00) failed at testbench.jz:25
  Cycle: 256
  Expected: 8'h00
  Actual:   8'hFF

  Relevant State:
    counter.count = 8'hFF
    counter.carry = 1'b0
  (Source: counter.jz:18 -> result of uadd operator)
```

9.2 Runtime Error Format

Runtime errors (z observation, floating net reads, combinational loops) produce:

```
RUNTIME ERROR: "Counter increments after reset release"
  z observed at testbench.jz:30
  Cycle: 12
  Signal: result
  Value:  8'bzzzz_zz01
  Bits [7:2] are z

  Trace:
    counter.data_bus = 8'bzzzz_zz01  (floating net, no active driver)
```

9.3 Test Summary

After all test cases in a testbench have executed, the simulator reports a summary:

```
Testbench: counter
  PASS: "Reset clears counter"
  PASS: "Single increment"
  FAIL: "Increment and Wrap"

Results: 2 passed, 1 failed, 3 total
```

```
Seed: 0xDEADBEEF
```

10. COMPLETE EXAMPLES

10.1 Module Under Test

```
@module counter
    PORT {
        IN  [1] clk;
        IN  [1] rst_n;
        OUT [8] count;
    }

    REGISTER {
        cnt [8] = 8'h00;
    }

    ASYNCHRONOUS {
        count <= cnt;
    }

    SYNCHRONOUS(CLK=clk RESET=rst_n RESET_ACTIVE=Low) {
        cnt <= cnt + 8'h01;
    }
@endmod
```

10.2 Testbench Example

```
@testbench counter
    CLOCK {
        clk;
    }

    WIRE {
        rst_n [1];
        count [8];
    }

    TEST "Reset holds counter at zero" {
        @new dut counter {
            clk [1] = clk;
            rst_n [1] = rst_n;
            count [8] = count;
        }

        @setup {
            rst_n <= 1'b0;
        }
```

```
        // Hold reset for 5 cycles
        @clock(clk, cycle=5)

        // Counter should remain at reset value
        @expect_equal(count, 8'h00)
    }

    TEST "Counter increments after reset release" {
        @new dut counter {
            clk [1] = clk;
            rst_n [1] = rst_n;
            count [8] = count;
        }

        @setup {
            rst_n <= 1'b0;
        }

        // Hold reset for 3 cycles
        @clock(clk, cycle=3)
        @expect_equal(count, 8'h00)

        // Release reset
        @update {
            rst_n <= 1'b1;
        }

        // Advance 1 cycle - first increment
        @clock(clk, cycle=1)
        @expect_equal(count, 8'h01)

        // Advance 4 more cycles
        @clock(clk, cycle=4)
        @expect_equal(count, 8'h05)
    }

    TEST "Counter wraps from FF to 00" {
        @new dut counter {
            clk [1] = clk;
            rst_n [1] = rst_n;
            count [8] = count;
        }

        @setup {
            rst_n <= 1'b0;
        }
```

```
        // Reset then release
        @clock(clk, cycle=1)
        @update {
            rst_n <= 1'b1;
        }

        // Advance 255 cycles to reach 0xFF
        @clock(clk, cycle=255)
        @expect_equal(count, 8'hFF)

        // One more cycle - should wrap
        @clock(clk, cycle=1)
        @expect_equal(count, 8'h00)
    }
@endtb
```

10.3 Memory Module Testbench

```
@module ram
    PORT {
        IN  [1] clk;
        IN  [1] rst_n;
        IN  [8] addr;
        IN  [8] wdata;
        IN  [1] wen;
        OUT [8] rdata;
    }

    MEM {
        mem [8] [256] = 8'h00 {
            OUT rd SYNC;
            IN  wr;
        };
    }

    SYNCHRONOUS(CLK=clk RESET=rst_n RESET_ACTIVE=Low) {
        mem.rd.addr <= addr;
        IF (wen) {
            mem.wr[addr] <= wdata;
        }
    }

    ASYNCHRONOUS {
        rdata <= mem.rd.data;
    }
@endmod
```

```
@testbench ram
    CLOCK {
        clk;
    }

    WIRE {
        rst_n [1];
        addr [8];
        wdata [8];
        wen [1];
        rdata [8];
    }

    TEST "Write then read" {
        @new dut ram {
            clk [1] = clk;
            rst_n [1] = rst_n;
            addr [8] = addr;
            wdata [8] = wdata;
            wen [1] = wen;
            rdata [8] = rdata;
        }

        @setup {
            rst_n <= 1'b0;
            wen <= 1'b0;
            addr <= 8'h00;
            wdata <= 8'h00;
        }

        // Reset
        @clock(clk, cycle=2)
        @update { rst_n <= 1'b1; }

        // Write 0xAB to address 0x10
        @update {
            addr <= 8'h10;
            wdata <= 8'hAB;
            wen <= 1'b1;
        }
        @clock(clk, cycle=1)

        // Disable write, set read address
        @update {
            wen <= 1'b0;
            addr <= 8'h10;
        }
```

```
        @clock(clk, cycle=1)

        // SYNC read has 1-cycle latency; data available now
        @expect_equal(rdata, 8'hAB)
    }
@endtb
```

11. RULES SUMMARY

11.1 Testbench Rules

Rule	Description
TB-001	`@testbench` must name a module that is in scope
TB-002	All module ports must be connected in the `@new` directive
TB-003	Port width in `@new` must match the module's declared port width
TB-004	`@new` right-hand side must be a testbench `CLOCK` or `WIRE` identifier
TB-005	`@setup` must appear exactly once per `TEST` block, after `@new` and before any other directives
TB-006	Any wire not assigned in `@setup` is implicitly initialized to `0`
TB-007	`@clock` clock identifier must refer to a declared `CLOCK`
TB-008	`@clock` cycle count must be a positive integer
TB-009	`@update` may only assign testbench `WIRE` identifiers
TB-010	`@update` may not assign clock signals
TB-011	`@expect_equal` / `@expect_not_equal` value width must match signal width
TB-012	Testbench must contain at least one `TEST` block
TB-013	Each `TEST` must contain exactly one `@new` instantiation
TB-014	`@expect` directives may not appear inside `@setup` or `@update` blocks
TB-015	If a CLOCK is not assigned in `@setup`, it implicitly initializes to `1'b0`
TB-016	All assignments within a single `@update` block are evaluated simultaneously
TB-017	All storage (REGISTER, LATCH, MEM) initializes with random bits
TB-018	`@expect` values must be fully determined (no `z` bits; `x` does not exist at runtime)
TB-019	Observing a signal containing `z` in an assertion is a runtime error (aborts test case)
TB-020	A file may not contain both RTL definitions (`@module`/`@project`) and verification constructs (`@testbench`)

Rule	Description
TB-021	`@expect_tristate` asserts all bits of the signal are `z`; it is the only assertion that accepts `z` values
PRT-001	Number of non-autonomous format specifiers in `@print` / `@print_if` must match the number of arguments
PRT-002	`@print` / `@print_if` may not appear inside `@setup` or `@update` blocks

11.2 Simulation Execution Rules

Rule	Description
SE-001	ASYNCHRONOUS logic must converge to a fixed point (no infinite combinational loops)
SE-002	SYNCHRONOUS updates use non-blocking assignment semantics (all RHS evaluated before any LHS updated)
SE-003	Immediate reset overrides pending synchronous updates
SE-004	Reading a net that resolves to `z` in a non-tristate context is a runtime error
SE-005	Storage initializes with random bits; reset values apply only after reset is asserted
SE-008	`x` is not a runtime value; it never appears during simulation
SE-006	Hierarchical names must be stable and deterministic across compilations
SE-007	Simulator must support reproducible runs via fixed random seed
SE-009	Given identical source and seed, all output must be bit-identical across runs

12. COMMAND-LINE INTERFACE

12.1 Testbench Execution

```
jz-hdl --test <file.jz>                      # Run all testbenches in file
jz-hdl --test <file.jz> --seed=<hex>         # Fixed random seed for reproducibility
jz-hdl --test <file.jz> --filter="<pattern>" # Run only tests matching pattern
jz-hdl --test <file.jz> --verbose             # Print all assertion results (pass and fa
```

12.2 Exit Codes

Code	Meaning
0	All tests passed
1	One or more test failures
2	Runtime error (x/z observation, combinational loop, etc.)
3	Compile error in testbench file